

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The complete implementation of the Single Layer Perceptron experiment, including the source code, dataset details, execution instructions, and dependencies, is available in the GitHub repository.

GitHub Repository:

<https://github.com/Teena-Pranava/Deep-Learning-Lab>

Task 7: Graphs

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

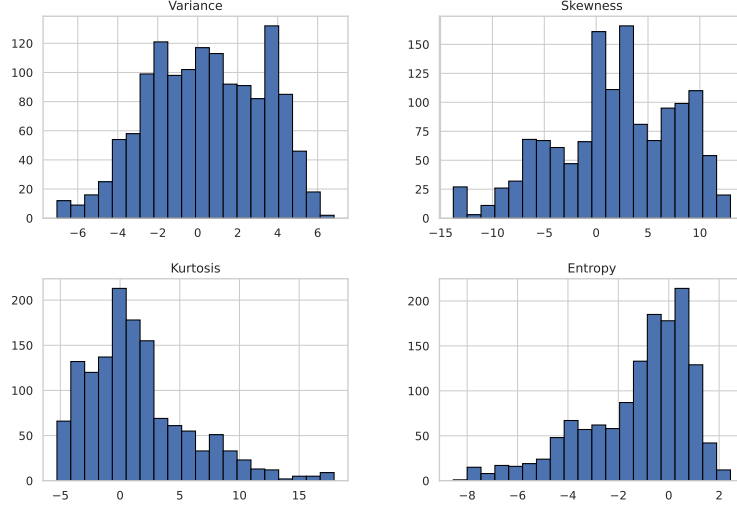


Figure 1: Feature Histograms

The histograms illustrate the distribution of values for each feature in the dataset. Variance and Skewness are spread over a wider range, showing higher variability, while Kurtosis and Entropy are comparatively more concentrated with a few extreme observations. The differences in these distributions indicate that each feature captures distinct characteristics of the banknotes, making them useful for the classification task.

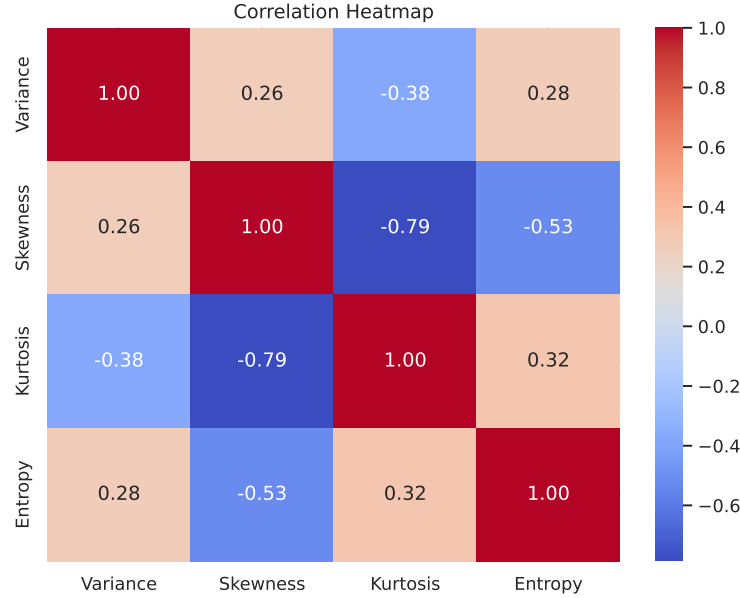


Figure 2: Correlation Heatmap

The correlation heatmap highlights the relationships between the input features and the target class. Variance has the strongest positive correlation with the class, whereas Skewness is negatively correlated. The moderate correlations among the features indicate that they provide complementary information, making them useful for building the classification model.

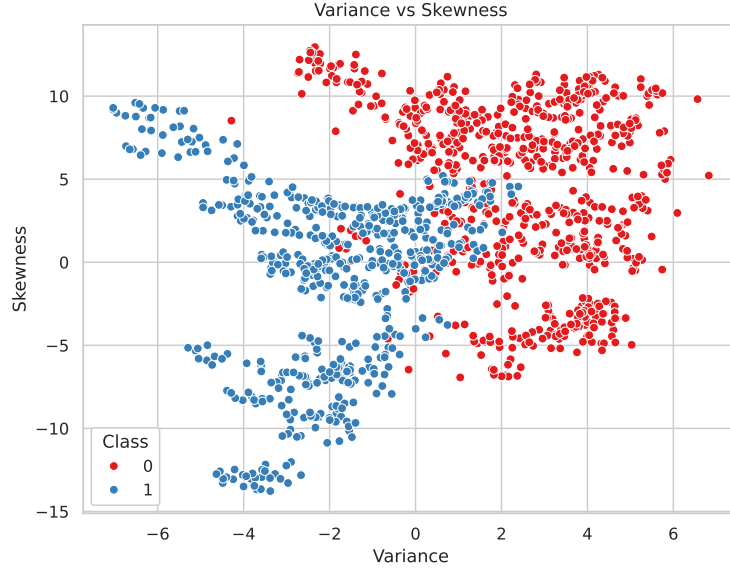


Figure 3: Scatter Plot

The scatter plot shows that the two classes form noticeable clusters when plotted using Variance and Skewness. Although there is some overlap between the classes, most genuine and forged banknotes occupy different regions of the feature space. This indicates that these features are effective in separating the two classes and are suitable for linear classification using a perceptron.

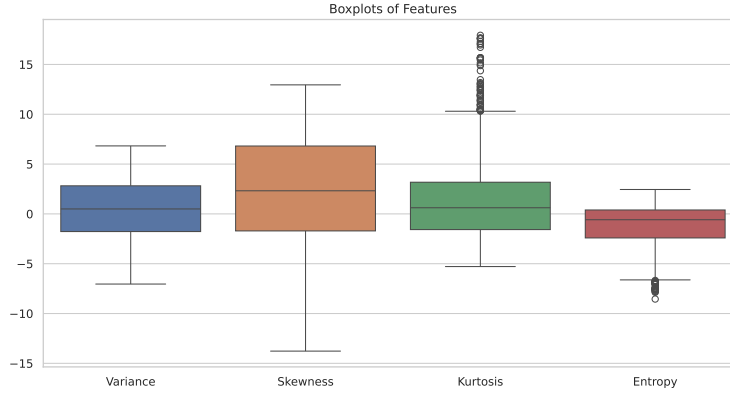


Figure 4: Box Plot

The boxplots provide a clear view of the spread and variability of each feature while highlighting potential outliers. Kurtosis contains several high-value outliers, whereas Entropy has a few low-value outliers. Since these observations are part of a genuine dataset and not due to measurement errors, they were retained for model training.

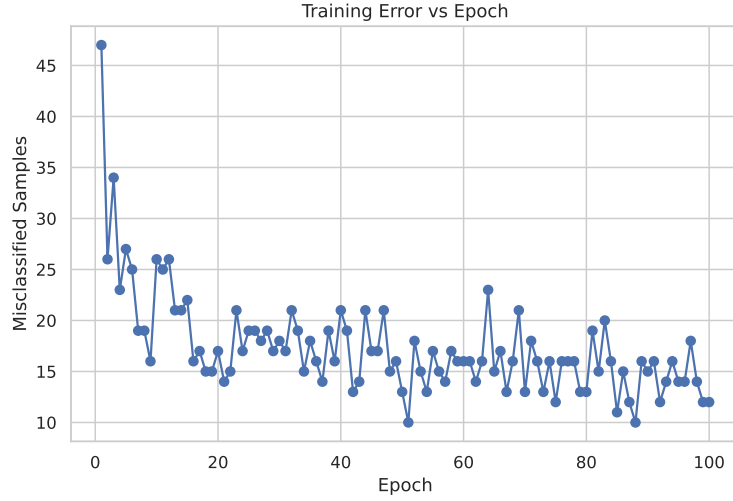


Figure 5: Training Error vs Epoch

The graph illustrates how the number of misclassified samples changes during training. A noticeable reduction in errors over successive epochs indicates that the perceptron is learning from its mistakes and improving its predictions. Once the error stabilizes, it suggests that the model has reached convergence and further training results in minimal improvement.

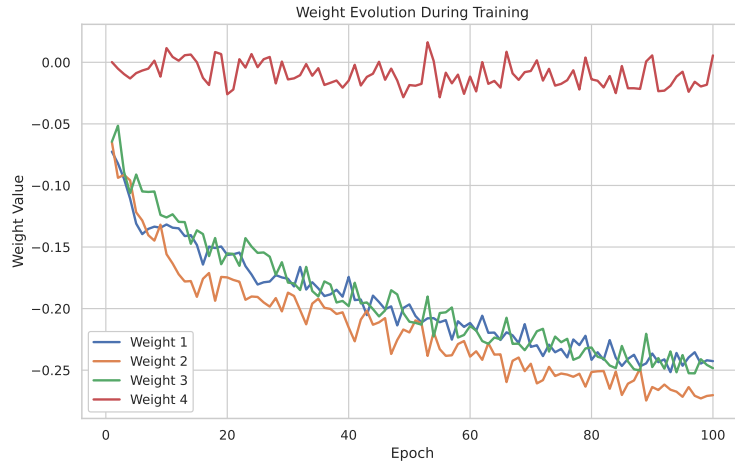


Figure 6: Weight Evolution

The weight evolution graph shows how the feature weights are adjusted throughout the training process. The weights change rapidly during the initial epochs as the model learns the underlying patterns in the data and gradually stabilize as training progresses. This stabilization indicates that the perceptron has converged to a suitable set of parameters.

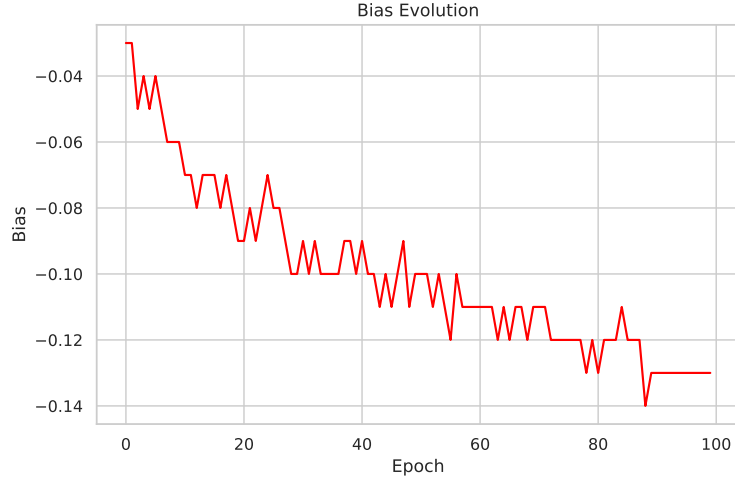


Figure 7: Bias Evolution

The bias changes continuously during training to improve the position of the decision boundary. Initially, larger updates are observed as the model learns from classification errors, but these updates become smaller over time. The final stable bias value indicates that the perceptron has reached a balanced decision boundary.

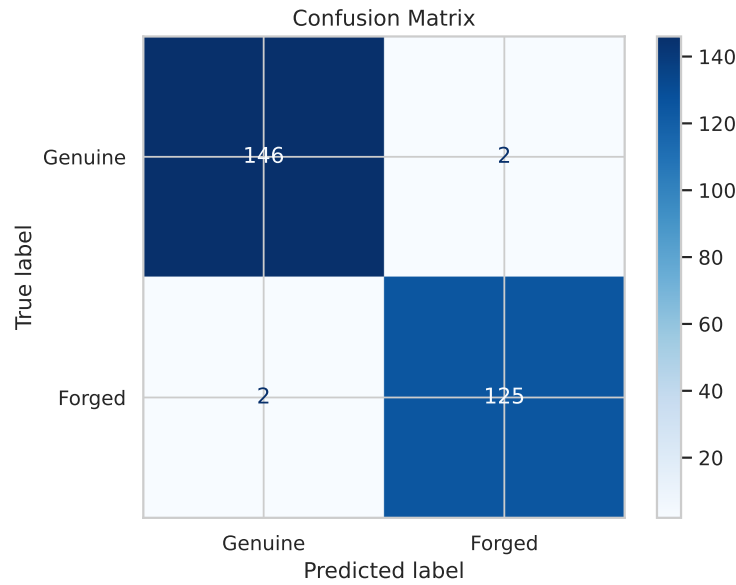


Figure 8: Confusion Matrix

The confusion matrix summarizes the classification performance of the perceptron by comparing the predicted labels with the actual labels. Most samples are correctly classified, as indicated by the high values along the main diagonal. The small number of misclassifications suggests that the model has learned an effective decision boundary for distinguishing between genuine and forged banknotes.

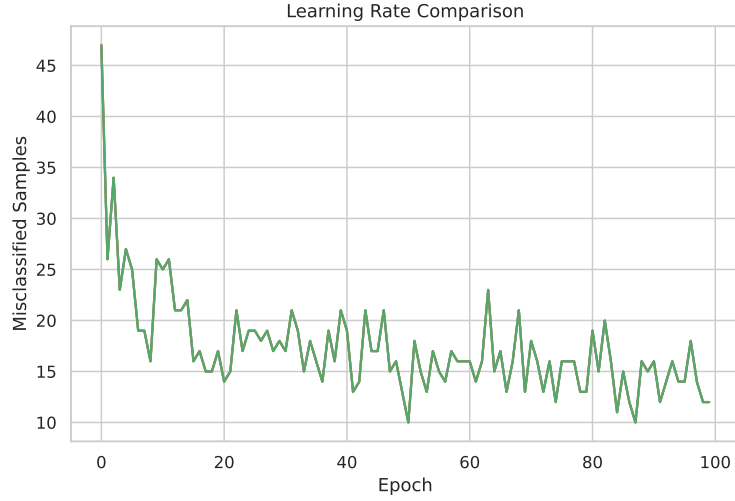


Figure 9: Learning Rate Comparison

The learning rate comparison demonstrates how different learning rates influence the training process. A smaller learning rate converges more slowly, requiring additional epochs to reduce the error, whereas a larger learning rate converges more quickly but may fluctuate before stabilizing. Among the tested values, a learning rate of 0.01 provides a good balance between convergence speed and training stability.

8. Performance Tables

Training Summary

Parameter	Value
Dataset Size	1372
Train/Test Split	80:20
Learning Rate	0.01
Epochs	100
Final Weights	[-0.2427, -0.2703, -0.2484, 0.0056]
Final Bias	-0.13
Accuracy	0.9855
Precision	0.9843
Recall	0.9843
F1-score	0.9843

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	47	-0.0725	-0.0649	-0.0645	0.0002	-0.03
2	26	-0.0822	-0.0938	-0.0515	-0.0052	-0.03
3	34	-0.0947	-0.0914	-0.0891	-0.0096	-0.05
4	23	-0.1107	-0.0958	-0.1063	-0.0131	-0.04
5	27	-0.1310	-0.1218	-0.0911	-0.0088	-0.05
6	25	-0.1395	-0.1284	-0.1049	-0.0067	-0.04
7	19	-0.1353	-0.1404	-0.1053	-0.0052	-0.05
8	19	-0.1335	-0.1448	-0.1049	0.0013	-0.06
9	16	-0.1343	-0.1318	-0.1240	-0.0116	-0.06
10	26	-0.1317	-0.1558	-0.1260	0.0115	-0.06
11	25	-0.1343	-0.1633	-0.1234	0.0044	-0.07
12	26	-0.1348	-0.1721	-0.1296	0.0013	-0.07
13	21	-0.1411	-0.1780	-0.1297	0.0057	-0.08
14	21	-0.1403	-0.1776	-0.1474	0.0063	-0.07
15	22	-0.1483	-0.1904	-0.1364	0.0000	-0.07
16	16	-0.1642	-0.1758	-0.1395	-0.0126	-0.07
17	17	-0.1496	-0.1711	-0.1574	-0.0185	-0.08
18	15	-0.1508	-0.1937	-0.1427	0.0083	-0.07
19	15	-0.1495	-0.1743	-0.1640	0.0067	-0.08
20	17	-0.1564	-0.1747	-0.1552	-0.0260	-0.09

9. Discussion

1. Which optimization method was used?

Randomized Search with 5-fold cross-validation was used to find the best combination of hyperparameters. It tested different combinations of the selected parameters and chose the one that achieved the highest cross-validation accuracy. This approach reduced the time required to search for a good model configuration.

2. Which hyperparameters were selected?

The best model used two hidden layers with 128 and 64 neurons, the RMSprop optimizer, ReLU activation function, a batch size of 64, and 20 training epochs. The learning rate used was 0.001, which is the default learning rate of the RMSprop optimizer.

3. Did optimization improve performance?

Yes. Hyperparameter optimization slightly improved the model's performance. The testing accuracy increased from 87.98% for the baseline model to 89.03% for the optimized model. The precision, recall, and F1-score also showed a small improvement, indicating better overall classification performance.

4. Which hyperparameter had the greatest impact?

The optimizer and the number of hidden neurons had the greatest impact on the model's performance. Using the RMSprop optimizer together with 128 and 64 neurons allowed the model to learn more effectively and achieve better accuracy.

5. Compare Grid Search and Randomized Search.

Grid Search evaluates every possible combination of hyperparameters, making it more time-consuming but more exhaustive. Randomized Search evaluates only a selected number of combinations, making it faster while still producing good results. In this experiment, Randomized Search was preferred because it reduced the search time without significantly affecting the model performance.

6. Recommend the best MLP configuration.

Based on the experimental results, the recommended MLP configuration consists of an input layer with 784 neurons, two hidden layers with 128 and 64 neurons, ReLU activation, RMSprop optimizer, a batch size of 64, 20 training epochs, and a Softmax output layer with 10 neurons. This configuration achieved the highest testing accuracy among the models evaluated.

1 10. Additional Task

Question

1. Code the perceptron learning algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built-in packages. (K4)

Decision Boundary Plots

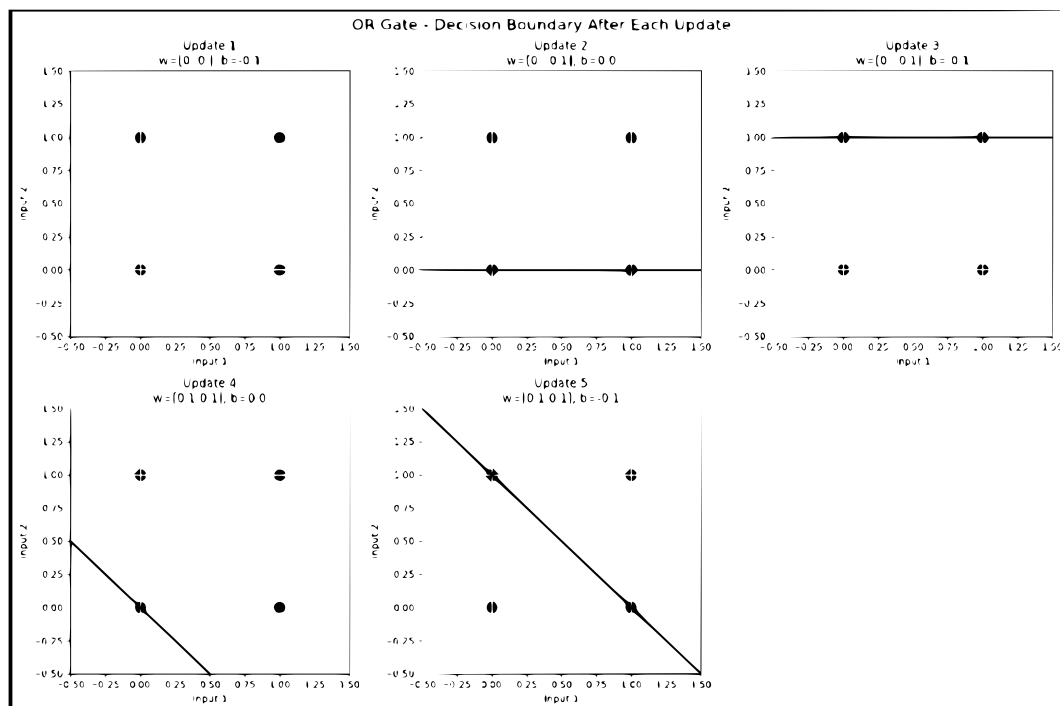


Figure 10: Decision boundary plots for the OR Gate after each weight update.

OR Gate :

Epoch 1

Update 1

Weights : $[0. \ 0.]$

Bias : -0.1

Update 2

Weights : $[0. \ 0.1]$

Bias : 0.0

Epoch 2

Update 3
Weights : [0. 0.1]
Bias : -0.1

Update 4
Weights : [0.1 0.1]
Bias : 0.0

Epoch 3

Update 5
Weights : [0.1 0.1]
Bias : -0.1

Epoch 4

Converged Successfully .

Final Weights : [0.1 0.1]
Final Bias : -0.1

Predictions :
[0 1 1 1]

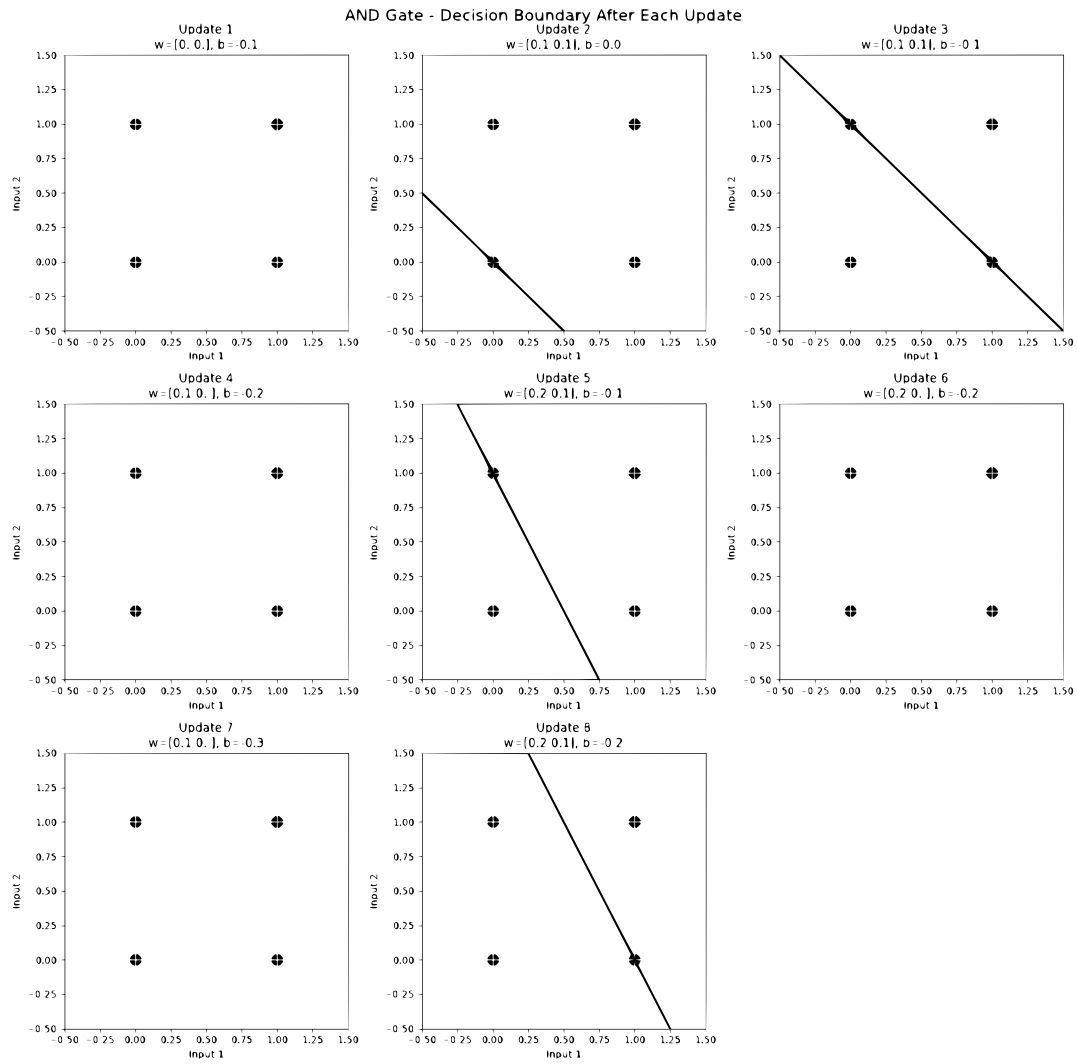


Figure 11: Decision boundary plots for the AND Gate after each weight update.

AND Gate :

Epoch 1

Update 1

Weights : $[0.0, 0.]$

Bias : -0.1

Update 2

Weights : $[0.1, 0.1]$

Bias : 0.0

Epoch 2

Update 3

Weights : [0.1 0.1]
Bias : -0.1

Update 4
Weights : [0.1 0.]
Bias : -0.2

Update 5
Weights : [0.2 0.1]
Bias : -0.1

Epoch 3

Update 6
Weights : [0.2 0.]
Bias : -0.2

Update 7
Weights : [0.1 0.]
Bias : -0.30000000000000004

Update 8
Weights : [0.2 0.1]
Bias : -0.20000000000000004

Epoch 4

Converged Successfully.

Final Weights : [0.2 0.1]
Final Bias : -0.20000000000000004

Predictions:
[0 0 0 1]

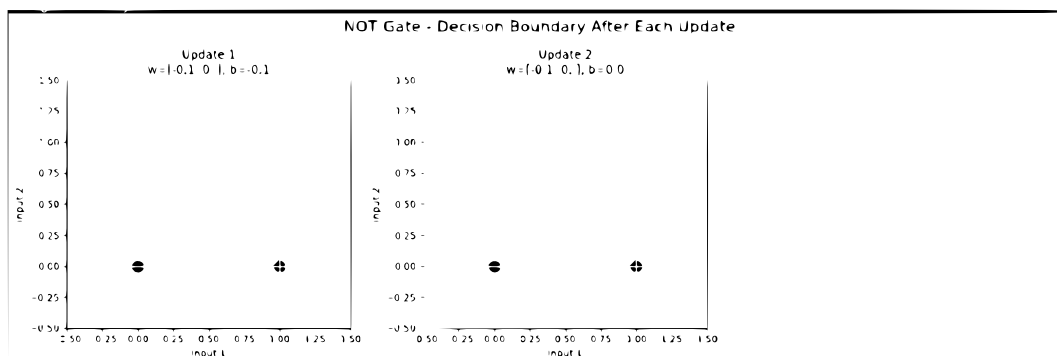


Figure 12: Decision boundary plots for the NOT Gate after each weight update.

NOT Gate :

Epoch 1

Update 1

Weights : [-0.1 0.]

Bias : -0.1

Epoch 2

Update 2

Weights : [-0.1 0.]

Bias : 0.0

Epoch 3

Converged Successfully.

Final Weights : [-0.1 0.]

Final Bias : 0.0

Predictions:

[1 0]

10. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.